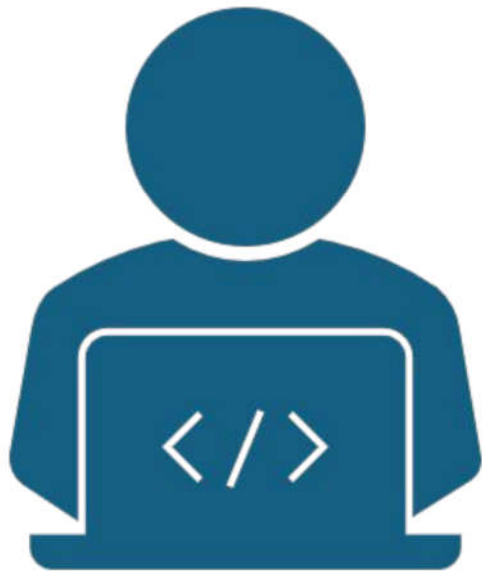




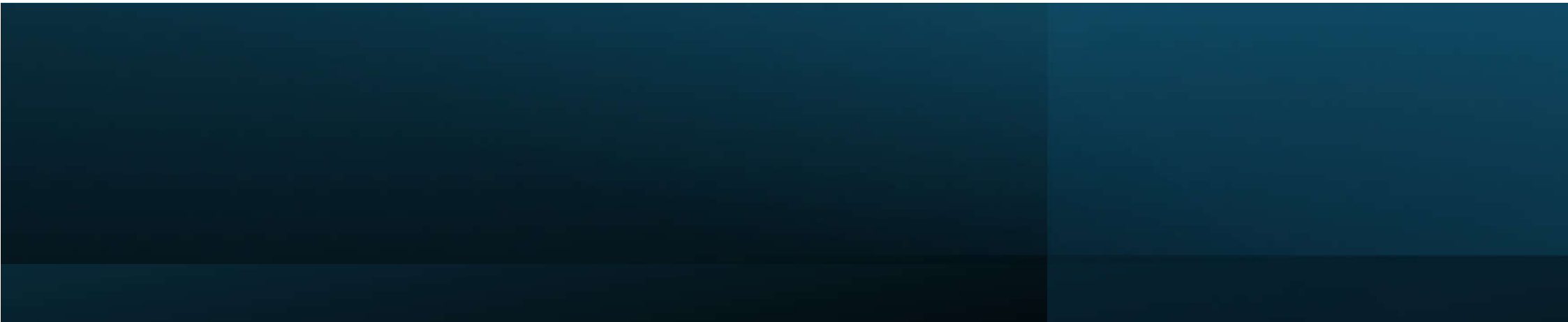
Basic Syntax

1. Variables

2. Data Types

3. Control Flow

Basic Syntax on CONTROL FLOW



Control Flow



Control flow is the order in which individual statements, instructions, or function calls are executed or evaluated.



The control flow of a Python program is regulated by conditional statements, loops, and function calls.

Different Aspects on Control Flow

Understanding Conditional Statements

Conditional Operators

IF Statement

IF-ELSE Statement

IF-ELIF=ELSE Statement

Nested Conditional Statements

NOT Keyword

Logical Operators

Collection Conditional Statements



Conditional Statements



Conditional Statements

- A statement that makes the program smarter, it makes the program decides on what to do in certain **CONDITIONS**.
- Conditional statements allow you to make decisions in your code, which can make your code more efficient and easier to read.
- **Conditional Statements**
 - IF Statement (1 Condition)
 - IF-ELSE Statement (2 Conditions)
 - IF-ELIF-ELSE Statement (3 or more Conditions)
 - NESTED Conditional Statements (Conditions after a Condition)

Conditional Operators

- Are used to compare values inside a **Conditional Statements**.
- **Conditional Operators**
 - == - EQUAL
 - != - NOT EQUAL
 - > - GREATER THAN
 - < - LESS THAN
 - >= - GREATER THAN or EQUAL
 - <= - LESS THAN or EQUAL

IF STATEMENT

- Used when dealing with ONE Condition
- If the condition is True, the code block indented below the if statement will be executed. If the condition is False, the code block will be skipped.

Syntax Example

```
if age >=18;  
    print("Legal Age")
```


IF – ELSE STATEMENT

- Used when dealing with TWO Conditions.
- The if-else statement is used to execute both the true part and the false part of a given condition. If the condition is true, the if block code is executed and if the condition is false, the else block code is executed.

Syntax Example

if age >=18:

print("Legal Age")

else:

print("Too Young")

IF – ELIF – ELSE STATEMENT

- Used when dealing with **THREE OR MORE** Conditions.
- The if-elif-else statement is used to conditionally execute a statement or a block of statements. Conditions can be true or false, execute one thing when the condition is true, something else when the condition is false.

Syntax Example

```
if age >= 18:  
    print("Legal Age")  
elif age >= 13:  
    print("Teenager")  
else:  
    print("Too Young")
```

NESTED CONDITIONAL STATEMENT

- Used when dealing with **CONDITIONS INSIDE A CONDITION**.
- The nested if statements are used to make decisions based on other decisions. Nesting of if statements mean to write an if condition inside another if condition.

Syntax Example

```
if age >= 18:  
    if height >= 170:  
        print("Legal Age and Tall")  
    elif height >= 156:  
        print("Legal Age and Average")  
    else:  
        print("Legal Age and Short")
```

NOT KEYWORD

- Used to **invert** the condition value.
- The not keyword is used in conditional statements or other Boolean expressions to flip the Boolean meaning or result.

Syntax Example

if not age >=18:

print("Too Young")



LOGICAL OPERATORS

- Used to include **2 or MORE CONDITIONS** in one line

LOGICAL OPERATORS

- AND – BOTH Condition must be true
 - OR – EITHER Condition must be true
-

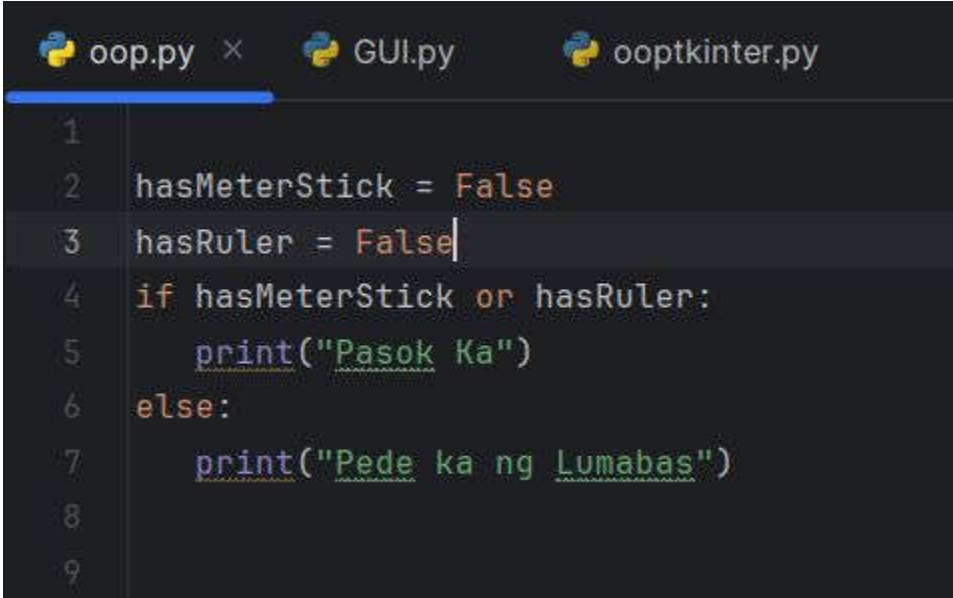
```
oop.py x GUI.py ooptkinter.py
1
2 age = int(input("Enter Your Age : "))
3 height = int(input("Enter Your Height : "))
4 if age >= 18 and height >= 176:
5     print("Tall and Legal Age")
6 elif age >= 18 and height >= 150:
7     print("Average and Legal Age")
8 elif age >= 18:
9     print("Short and Legal Age")
10 else:
11     print("Too Young")
12 print("Thank You!!!")
13
14
```

AND Logical Operators

Example

OR Logical Operator

- Example



```
oob.py × GUI.py ooptkinter.py
1
2 hasMeterStick = False
3 hasRuler = False
4 if hasMeterStick or hasRuler:
5     print("Pasok Ka")
6 else:
7     print("Pede ka ng Lumabas")
8
9
```

COLLECTION CONDITIONAL STATEMENTS

- Used to **check** an item if its in a collection (list and tuples)

Syntax Example

```
bag = ["wallet", "gun", "handkerchief"]
```

```
if "gun" in bag:
```

```
    print("Contraband")
```

```
else:
```

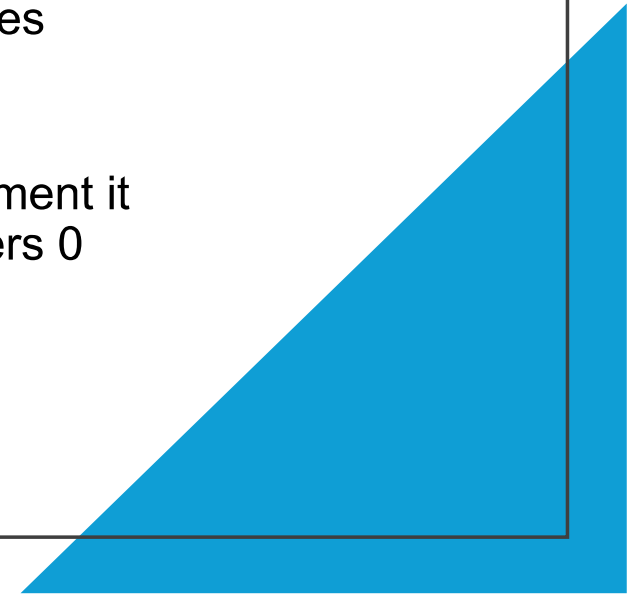
```
    print("All Clear")
```



**WHILE
Loop**

WHILE Loop

- **While Loop** is used to execute a block of statements repeatedly until a given condition is satisfied. When the condition becomes false, the line immediately after the loop in the program is executed.
- The loop starts with 'i' as 0 and continues to print 'i' and increment it by 1 until 'i' is no longer less than 5. This results in the numbers 0 through 4 being printed.





WHILE Loop

Syntax Example

```
age = 12
```

```
while (age < 18):
```

```
    print ("Still Young: " + str(age))
```

```
    age = age + 1
```



ELSE in WHILE Loop

- ELSE is added to the bottom of a while loop so that it can execute code when the loop is done.
 - Code to be executed when the condition is true is indented under the while block. The 'else' keyword, followed by another colon (:), starts the else block. This block will execute once the while condition becomes false.
-

ELSE in WHILE Loop

Syntax Example

age = 12

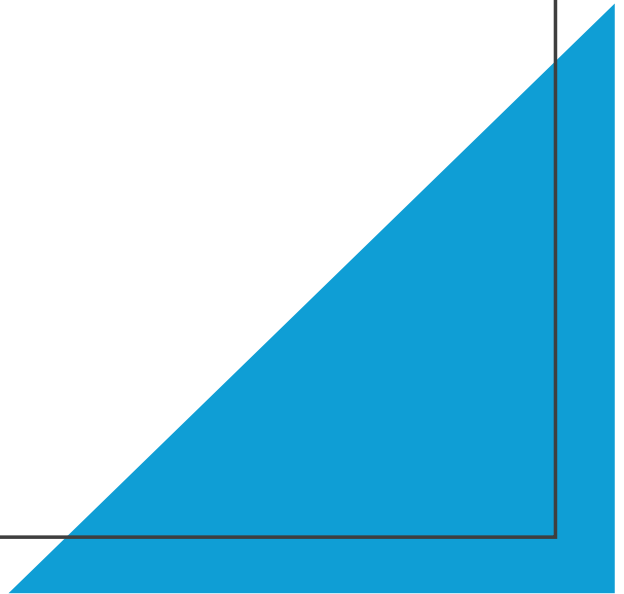
while age < 18:

print("Still Young: " + str(age))

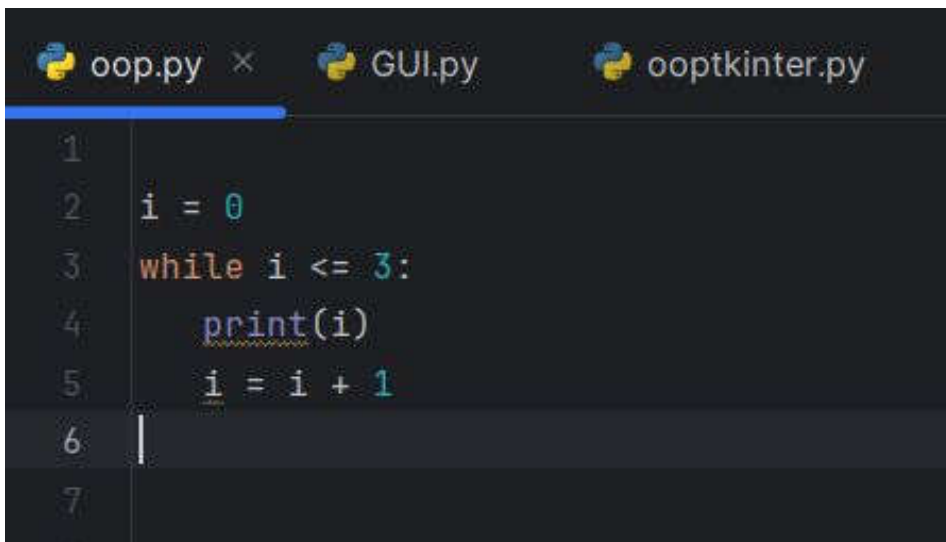
age = age + 1

else:

print("Legal Age: " + str(age))



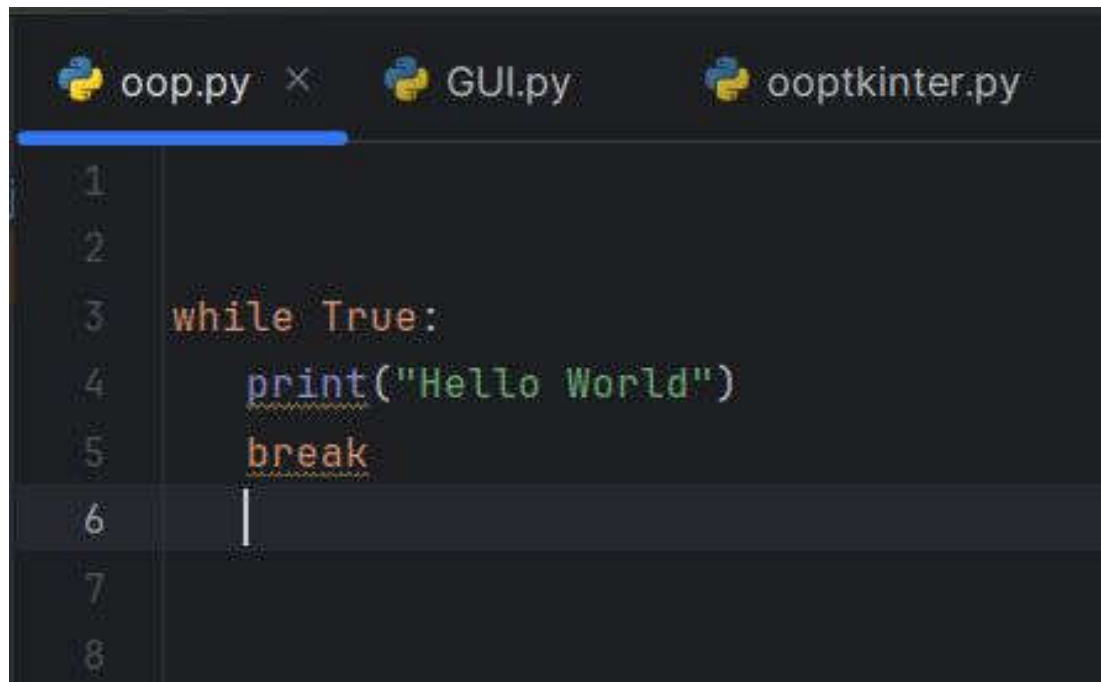
WHILE Loop in COLLECTIONS

A screenshot of a Python IDE with three tabs: 'oop.py', 'GUI.py', and 'ooptkinter.py'. The 'oop.py' tab is active and shows a while loop. The code is as follows:

```
1  
2 i = 0  
3 while i <= 3:  
4     print(i)  
5     i = i + 1  
6  
7
```

- **WHILE Loop** can be used to access every item in a **COLLECTION (Lists & Tuples)** Since it is **indexed and ordered**.
- The while loop checks if the count is less than or equal to 5.
- As long as this particular condition holds true, the code block inside the loop is then executed.

BREAK Keyword in WHILE Loop



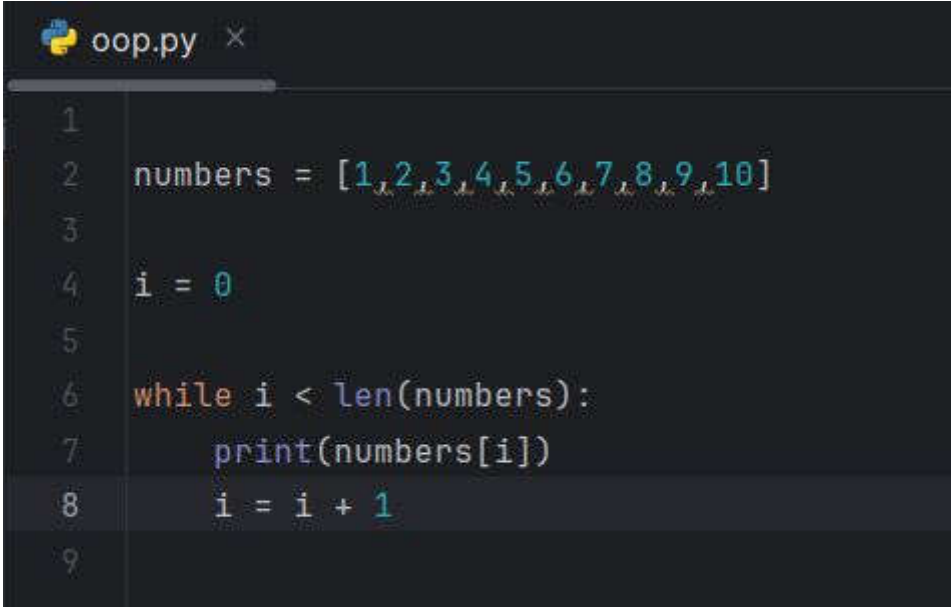
The screenshot shows a Python IDE with three tabs: 'oop.py', 'GUI.py', and 'ooptkinter.py'. The 'oop.py' tab is active. The code in the editor is as follows:

```
1  
2  
3 while True:  
4     print("Hello World")  
5     break  
6  
7  
8
```

- BREAK Keyword is used to stop the loop no matter what the condition is.
- 'Break' in Python is a loop control statement. It is used to control the sequence of the loop. Suppose you want to terminate a loop and skip to the next code after the loop; break will help you do that.

CONDITIONS in WHILE Loop

- You can use any Conditional Statement inside a WHILE Loop.
- The loop starts with 'i' as 0 and continues to print 'i' and increment it by 1 until 'i' is no longer less than 5. This results in the numbers 0 through 4 being printed.



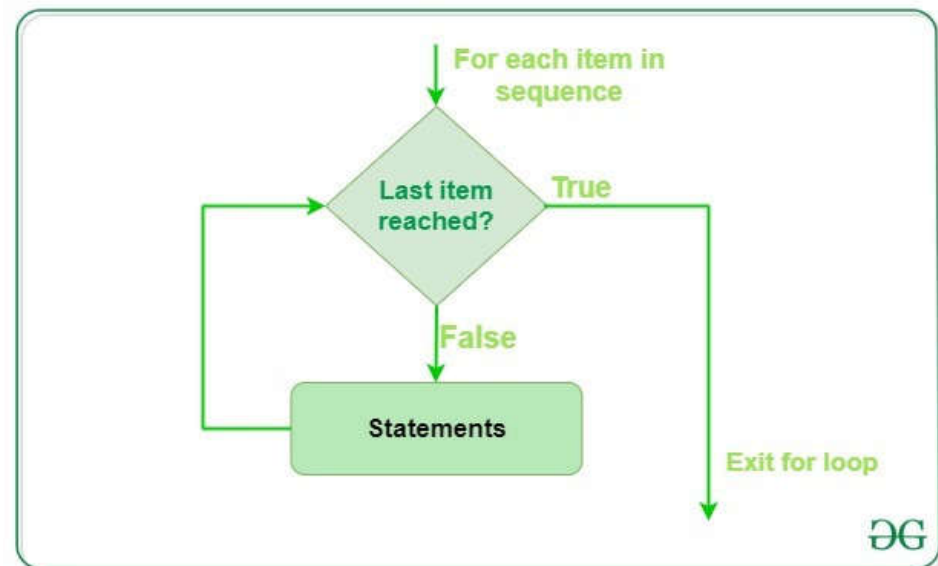
```
oop.py x
1
2 numbers = [1,2,3,4,5,6,7,8,9,10]
3
4 i = 0
5
6 while i < len(numbers):
7     print(numbers[i])
8     i = i + 1
9
```




FOR Loop

FOR Loop

- A for loop is used to iterate over a sequence (list, tuple, set, dictionary, and string).
- This is less like the for keyword in other programming languages, and works more like an iterator method as found in other object-orientated programming languages.
- The loop iterates as many times as the number of elements and prints the elements serially.



FOR Loop

- A statement that is commonly used to iterate through a collection or to execute a block of code in a certain amount of times

A screenshot of a Python IDE with two tabs: 'oop.py' and 'GUI.py'. The 'oop.py' tab is active and shows a list 'l' containing the strings 'geeks', 'for', and 'geeks'. A for loop iterates over this list, printing each element. The code is as follows:

```
1  
2 l = ["geeks", "for", "geeks"]  
3  
4 for i in l:  
5     print(i)  
6
```

FOR Loop in COLLECTIONS

- FOR Loop can be used to access every item in a COLLECTION (Lists & Tuples) in a very easy way.

Syntax Example

```
for x in cars = ["Honda", "Toyota", "Hyunda"]  
    for car in cars:  
        print(cars)
```

ELSE in FOR Loop

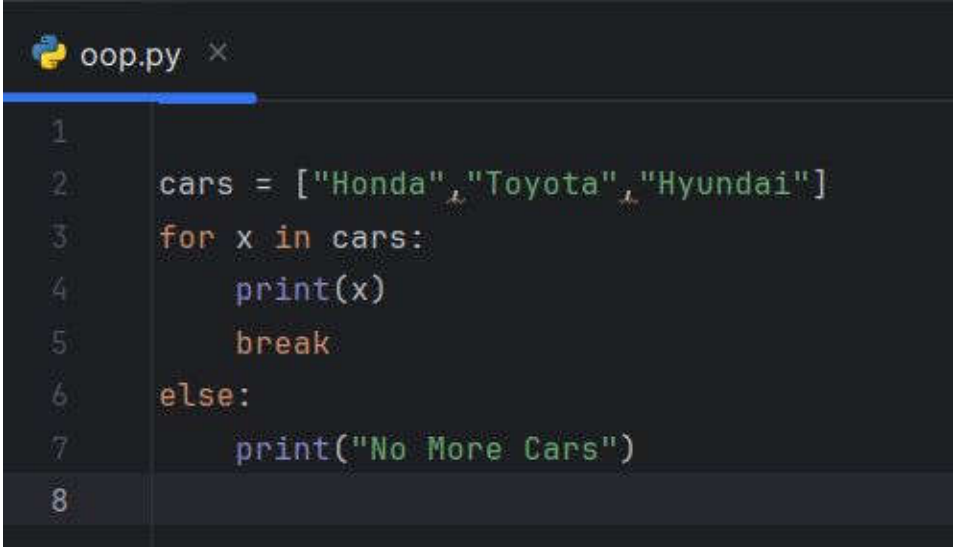
- ELSE is added to the bottom of a for loop so that it can execute code when the loop is done.
- Python supports an else clause in for and while loops. The else body is executed only if no break is encountered during the execution of the associated loop body.

Syntax Example

```
cars = ["Honda", "Toyota", "Hyundai"]  
    for car in cars:  
        print(car)  
    else:  
        print("No More Fruits")
```

BREAK Keyword in FOR Loop

- BREAK Keyword is used to stop the loop earlier than its supposed to finish.
- You can use a break statement with both for loops and while loops.
- In a nested loop, break will stop execution of the innermost loop. The flow of the program resumes at the next line of code immediately after the block.



```
oop.py x
1
2 cars = ["Honda", "Toyota", "Hyundai"]
3 for x in cars:
4     print(x)
5     break
6 else:
7     print("No More Cars")
8
```

CONDITIONS in FOR Loop

You can use any Conditional Statement inside a FOR Loop.

```
oop.py x
1
2 cars = ["Honda", "Toyota", "Hyundai"]
3 for x in cars:
4     print(x)
5     if x == "Toyota":
6         break
```

RANGE() in FOR Loop

- Loops a set of code in a specified number of times.

Syntax

```
for x in range(y):
```

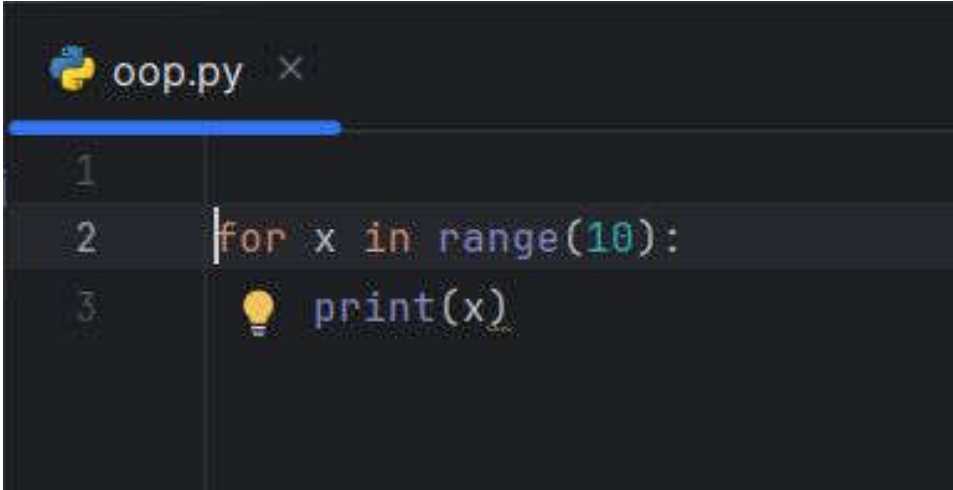
```
range(5):
```

```
#Do Something
```

Example

```
for x in
```

```
print(x)
```

A screenshot of a Python IDE window titled 'oop.py'. The code editor shows three lines of code: line 1 is empty, line 2 is 'for x in range(10):', and line 3 is 'print(x)'. A lightbulb icon is next to the print statement, indicating a suggestion or tip. The code is syntax-highlighted with 'for' in orange, 'x' in blue, 'in' in orange, 'range' in blue, '10' in green, and 'print' in blue.

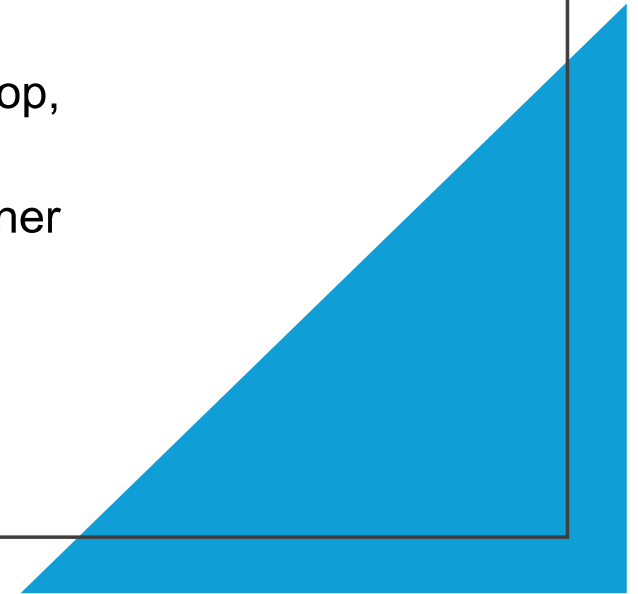
```
1  
2 for x in range(10):  
3     print(x)
```




NESTED FOR Loop

NESTED FOR Loop

- A nested loop refers to a loop within a loop, an inner loop within the body of an outer one.
- Further, the first pass of the outer loop will trigger the inner loop, which will execute to completion.
- After that, the second pass of the outer loop will trigger the inner loop again.

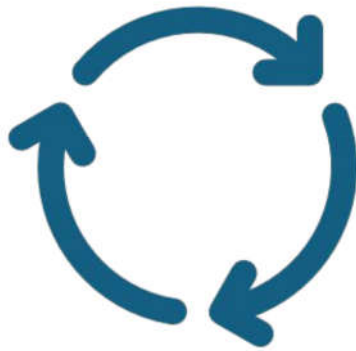


NESTED FOR Loop

- A For Loop inside a For Loop commonly used to iterate through a multi-dimensional collection.

```
oop.py x GUI.py
1
2 taste = ["red", "big", "tasty"]
3 identity = ["Head", "Bird", "Biggle"]
4
5 for x in taste:
6     for y in identity:
7         print(x, y)
8 |
```

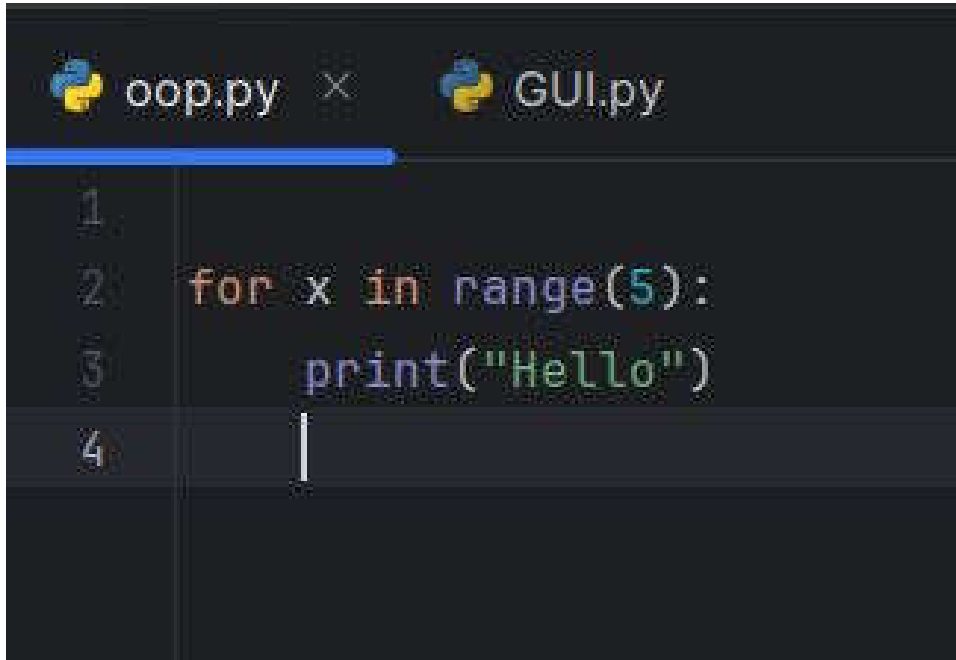
Range in Nested For Loops



- You can use nested for loops with the range() function to get the first 10 multiples of the specified range of numbers.
- First, specify the range of numbers (these numbers multiples we want to get) in the outer loop and specify a range of 10 numbers in the inner loop.

Using Range in Nested For Loops

- Example



The screenshot shows a code editor with two tabs: 'oop.py' and 'GUI.py'. The 'oop.py' tab is active and contains the following Python code:

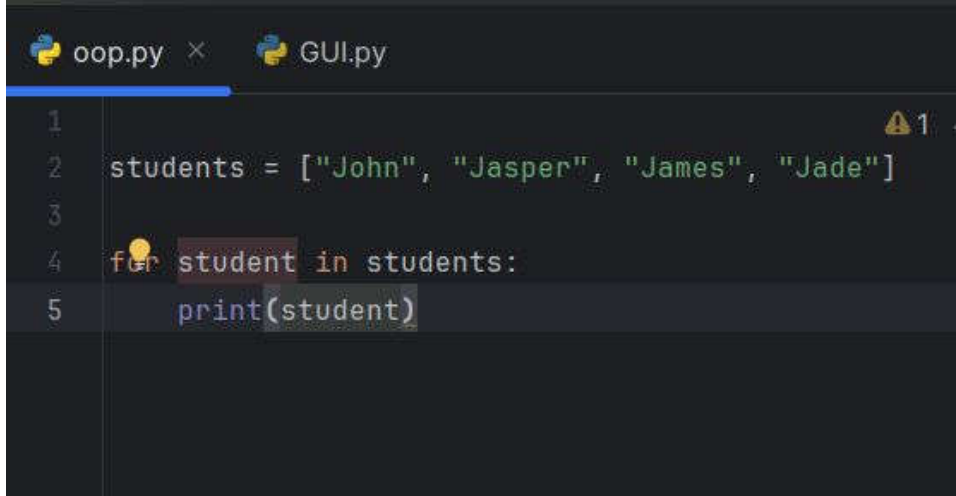
```
1  
2 for x in range(5):  
3     print("Hello")  
4 |
```

The code is displayed with syntax highlighting: 'for' is blue, 'x' is red, 'in' is blue, 'range(5)' is blue, 'print' is blue, and 'Hello' is green. A vertical cursor is positioned at the end of line 4.

Reading Multi-Dimensional Collection Using Nested For Loop

- The final **loop** has the size of the outer **loop**, and the numbers of elements in it of the inner **loop** Let me give you an **example** with your 2 lists.

Example – image at the otheside



```
oop.py x GUI.py
1
2 students = ["John", "Jasper", "James", "Jade"]
3
4 for student in students:
5     print(student)
```

